



Beyond the Scheduler

Designing Predictable Embedded Systems in
Modern C++

ELBERT DOCKERY



20
26



September 12 - 18

Beyond the Scheduler

Designing an embedded filesystem in C++20 for bounded work



Build it, or use what already exists?

The first question was simple: do I need another filesystem?

USE WHAT EXISTS

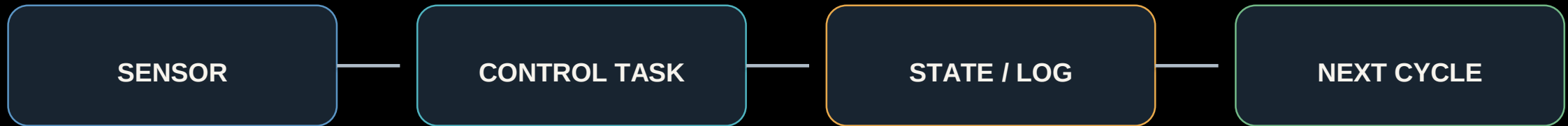
- Mature behavior
- Known ecosystem
- Lower implementation risk

BUILD TO LEARN

- Expose assumptions
- Control the constraints
- Measure what varies

I built a small filesystem to make those assumptions visible — not to replace every existing filesystem.

Storage sits underneath the real-time path

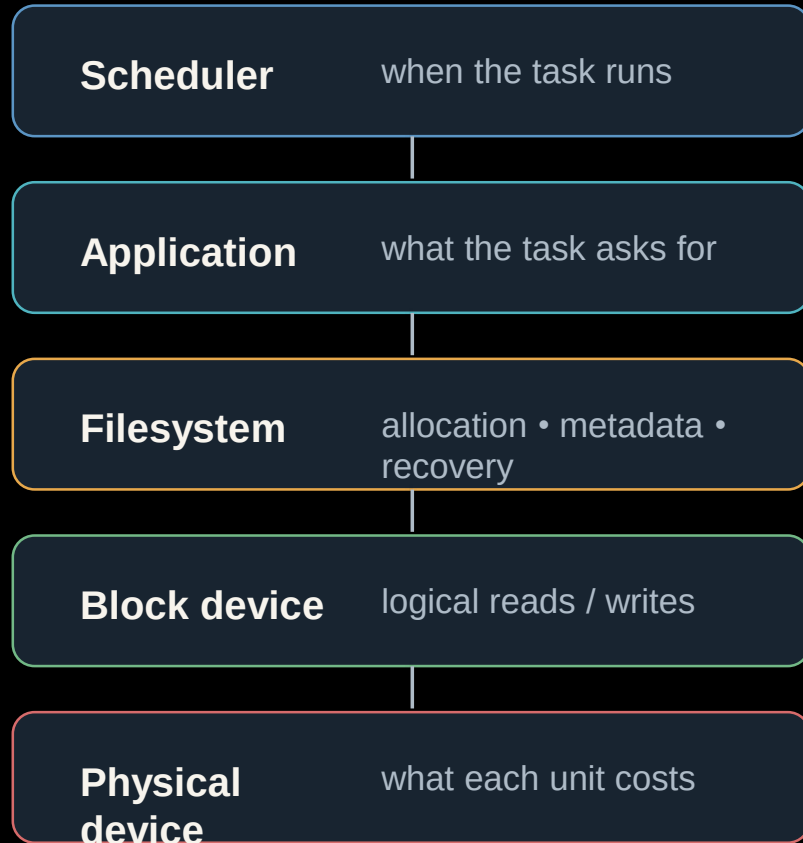


Configuration • calibration • fault state • checkpoints • logs

Can I explain and constrain the work storage introduces?

Robotics • aerospace • medical • industrial control • transportation

Where does predictability stop?



? How many candidates?

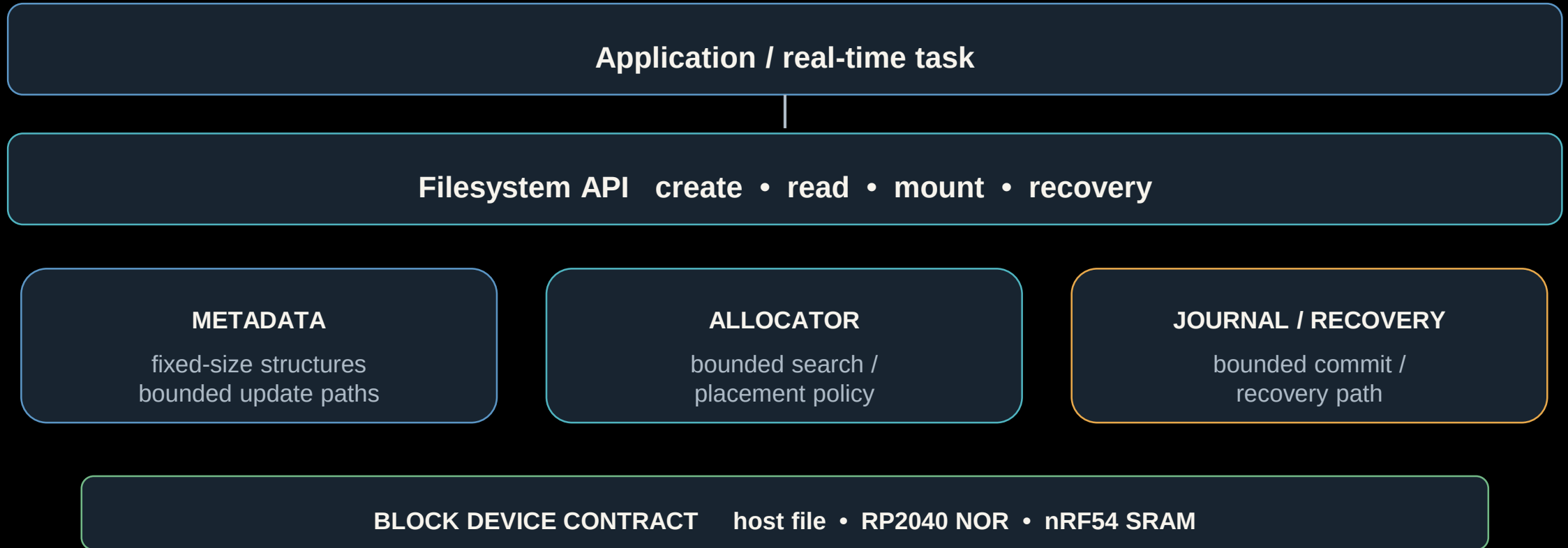
? How much metadata?

? How many block writes?

? How much recovery work?

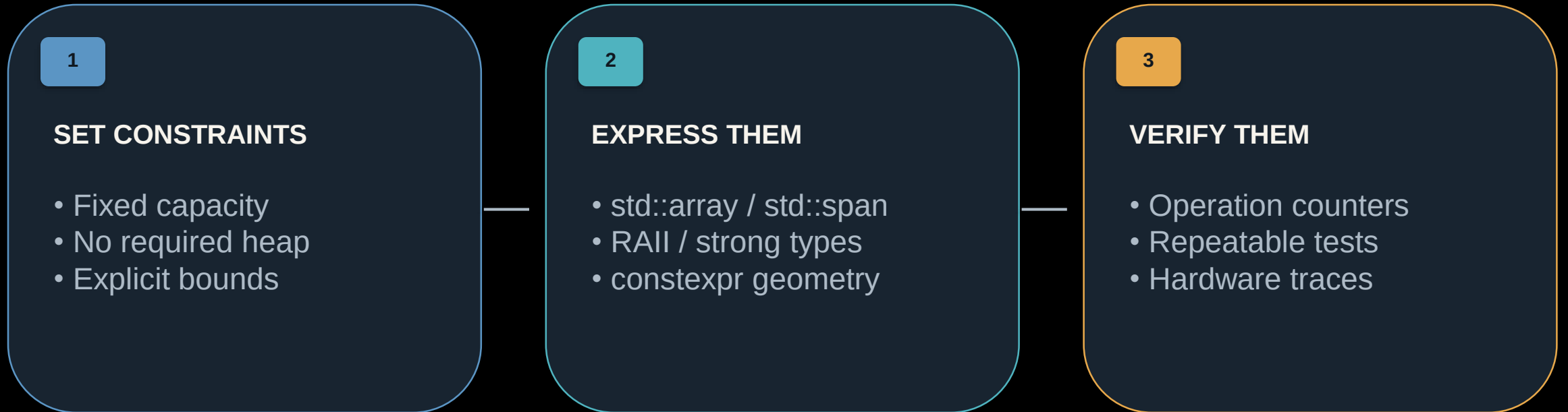
Real-time reasoning should not stop at the scheduler.

Make the work visible in the architecture



Bound filesystem work first. Characterize platform cost second.

C++20 makes the constraints explicit



The architecture creates the bounds. C++20 helps encode them clearly.

One byte changes the requested work

4096 bytes

1 data block

4097 bytes

2 data blocks

$$\text{Candidates}_{ax} = D - K + 1$$

$$\text{Inspections}_{ax} \leq \text{Candidates}_{ax} \times K$$

Observed logical work in the boundary experiment

4096 B

1 read / 9 writes

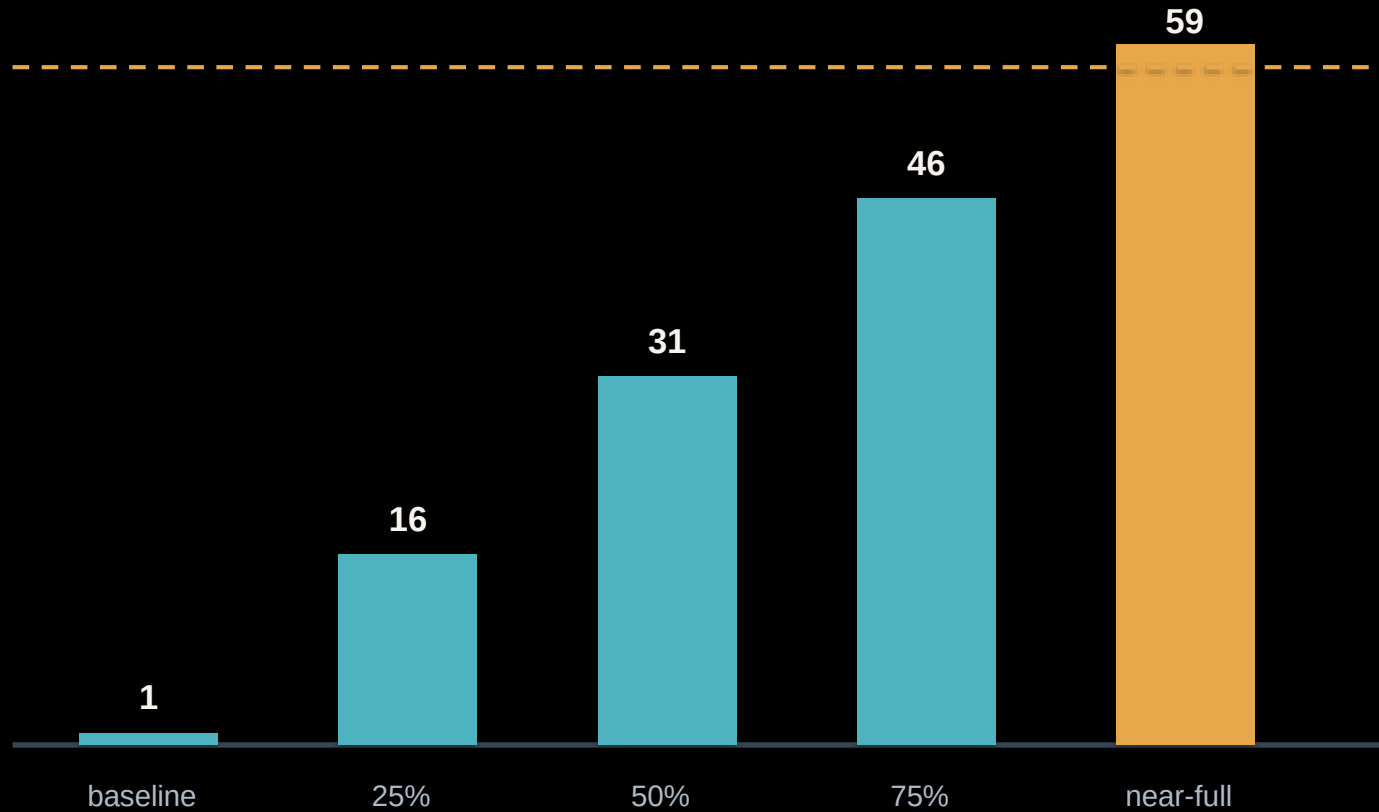
4097 B

1 read / 10 writes

**Prediction first.
Measurement second.**

Explain the requested work first.
Then measure what that work costs.

State changes the work. Configuration constrains it.



1,900 filesystem
measurements

19 workload groups

0 work-bound
violations

Bounded work exposed a separate memory problem

create_file() compiler-reported stack frame

12,536 B

————

264 B

~97.9%
reduction

What changed?

Block-sized scratch storage moved into one explicit reusable workspace.

The tradeoff

FileSystem object
~4.9 KiB → ~9.0 KiB

Memory did not disappear.
It became explicit and reusable.

Predictability also means making memory demand explicit.

Test the work model from several directions

	BOUND	REPEAT	PRESSURE	RECOVERY	HARDWARE
create / read	✓	✓	✓	●	✓
allocation	✓	✓	✓	●	✓
metadata	✓	✓	✓	✓	✓
journal / recovery	✓	✓	●	✓	✓

132 / 132

normal

132 / 132

ASan / UBSan

CI PASS

release gate

Same logical work. Different platform cost.

RP2040 • onboard NOR



4096 B 1R / 9W 384.541 ms
4097 B 1R / 10W 430.874 ms

nRF54L15 • SRAM backend



4096 B 1R / 9W 3.955 ms
4097 B 1R / 10W 4.277 ms

Invariant in this boundary case: 4096 → 1R / 9W • 4097 → 1R / 10W

Not a performance benchmark: NOR and SRAM have different storage semantics. These are observed times, not WCET.

What the experiment taught me

THREE TAKEAWAYS

1

Bound work before timing it.

Timing tells you what happened; a work model explains why.

2

Make resource demand explicit.

Stack, workspace, I/O and recovery are architectural resources.

3

Predictability is layered.

Scheduler → software work → platform cost.

NEXT

- Same external SPI NOR across MCUs
- MCU-specific RAM budgets
- Torn-write / brownout-safe persistence
- Platform timing contracts toward WCET
- Apply work contracts beyond storage

Scheduler: when work runs

Architecture: how much work

Platform: what the work costs

Predictability means explaining what can vary, constraining how much it can vary, and connecting that work to the platform.

THANK YOU!